

PDC Assignment 1

Parallelising Matrix Multiplication



Name: Ahmad Bin Rashid

Roll No: 2023-CS-53

Instructor: Sir Waqas Ali

Course: Distributed and Parallel Computing

University of Engineering and Technology

Contents

1	Problem Description	2
1.1	Problem Statement	2
1.2	Problem Attributes	2
1.3	Why Matrix Multiplication?	2
2	Parallelisation Design	2
2.1	Chosen Approach: MPI Collectives	2
2.2	Parallelism Pattern	3
2.3	Data Distribution Strategy	3
2.4	Communication Analysis	3
2.5	Synchronisation	3
2.6	Algorithm Outline	4
3	Implementation Details	4
3.1	Code Structure	4
3.2	Key Functions	4
3.3	Compilation and Execution	5
3.4	Core Parallel Code	5
3.5	Verification Method	6
4	Performance Analysis	6
4.1	Timing Results	6
4.2	Performance Plots	7
4.3	Analysis Questions	8
4.3.1	Q1: Is the speedup linear? If not, what limits it?	8
4.3.2	Q2: Efficiency at highest process count	8
4.3.3	Q3: Fraction of time in communication	9
4.3.4	Q4: Is the work evenly distributed?	9
4.3.5	Q5: Comparison with theoretical expectation	9
5	Reflection	9
5.1	Challenges Faced	9
5.2	Lessons Learned	10
6	References & AI Disclosure	10
6.1	References	10
6.2	AI Disclosure	10

1 Problem Description

1.1 Problem Statement

Matrix multiplication is one of the most fundamental operations in numerical computing. Given two $N \times N$ matrices A and B , the goal is to compute $C = A \times B$, where each element is defined as:

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \cdot B[k][j] \quad \text{for } 0 \leq i, j < N \quad (1)$$

1.2 Problem Attributes

Table 1: Problem summary

Attribute	Details
Problem Name	Matrix Multiplication ($C = A \times B$)
Original Course	Linear Algebra / Numerical Methods (previous semester)
Language	C (sequential), C + MPI (parallel)
Input Format	Two $N \times N$ matrices of random <code>double</code> ; N via <code>-n</code> flag
Output Format	$N \times N$ result matrix C ; timing and speedup to <code>stdout</code>
Sequential Complexity	$O(N^3)$
Parallel Complexity	$O(N^3/P)$ computation + $O(N^2)$ communication

1.3 Why Matrix Multiplication?

- It is computationally intensive ($O(N^3)$), so parallelisation yields meaningful speedups for large N .
- The data-access pattern is perfectly regular, making row-scatter distribution straightforward.
- It appears in real applications: deep learning, physics simulations, and graph algorithms.
- Results are easy to verify against the sequential version numerically.

2 Parallelisation Design

2.1 Chosen Approach: MPI Collectives

MPI Collectives were selected over Pthreads and MPI point-to-point for three reasons.

First, the data distribution is perfectly regular. Each process receives exactly $\lfloor N/P \rfloor$ contiguous rows of matrix A , making `MPI.Scatter` the natural fit.

Second, all processes need the complete matrix B , which is most efficiently delivered via a single `MPI_Bcast` call. The MPI library uses an optimised tree-based algorithm internally, outperforming a manual point-to-point loop.

Third, MPI scales beyond a single shared-memory node, unlike Pthreads. MPI point-to-point would require manual loop logic to distribute rows and collect results, adding code complexity with no performance benefit over `MPI_Scatter/Gather`.

2.2 Parallelism Pattern

The pattern is **data parallelism** with an **embarrassingly parallel** computation phase. Once each process has its slice of A and the full matrix B , the local multiplication requires zero inter-process communication until the final gather. There are no loop-carried dependencies across row boundaries.

2.3 Data Distribution Strategy

Table 2: Communication steps

#	Operation	Description
1	<code>MPI_Bcast(B)</code>	Root broadcasts all N^2 elements of B to every process
2	<code>MPI_Scatter(A)</code>	Root scatters $\lfloor N/P \rfloor \times N$ rows of A to each rank
3	Local multiply	Each rank computes its N/P rows of C independently
4	<code>MPI_Gather(C)</code>	Root gathers result rows from each rank into full C

2.4 Communication Analysis

Total data transferred (in bytes):

$$\begin{aligned}
 \text{Bcast}(B) &= 8N^2 \text{ bytes} \\
 \text{Scatter}(A) &= 8N^2 \text{ bytes} \\
 \text{Gather}(C) &= 8N^2 \text{ bytes} \\
 \text{Total communication} &= 24N^2 \text{ bytes}
 \end{aligned}$$

For $N = 1024$ this is ≈ 25 MB — small compared to $O(N^3/P)$ computation. The computation-to-communication ratio grows as N increases, so larger problems benefit more from parallelism.

2.5 Synchronisation

No explicit mutex or barrier is needed during the computation phase because each process operates on independent memory. The implicit synchronisation points are the collective

calls themselves: `MPI_Bcast` blocks until all processes have received B , and `MPI_Gather` blocks the root until all rows are collected. An explicit `MPI_Barrier` is used only for accurate timing.

2.6 Algorithm Outline

```
MPI_Init()
if rank == 0: fill A, B with random doubles (seed = 42)

MPI_Bcast(B, N*N)           // all ranks receive full B
MPI_Scatter(A -> A_local)    // each rank gets N/P rows of A

for i in 0 .. rows_per_proc:
    for j in 0 .. N:
        C_local[i][j] = dot(A_local[i][:], B[:,j])

MPI_Gather(C_local -> C)     // root assembles full result C
if rank == 0: print timing, C[0][0]
MPI_Finalize()
```

Listing 1: High-level parallel algorithm

3 Implementation Details

3.1 Code Structure

Table 3: Repository file listing

File	Description
sequential/sequential.c	Baseline $O(N^3)$ implementation
sequential/Makefile	Build rules for sequential version
parallel/parallel.c	MPI Collectives parallel version
parallel/Makefile	Build rules for parallel version
results/timing_data.csv	Timing data across sizes and processes
results/speedup_plot.png	Speedup vs processes chart
results/scaling_plot.png	Time vs problem size chart
results/efficiency_plot.png	Efficiency vs processes size chart
report/report.pdf	This document

3.2 Key Functions

`fill_random(double *M, int N)`

Fills the $N \times N$ matrix with uniform random double values. Seed is fixed at 42 for reproducibility across runs.

`sequential_multiply(A, B, C, N)`

Standard triple-loop $O(N^3)$ baseline. Also called for correctness checking when $N \leq 64$.

`main()` in `parallel.c`

Handles argument parsing, MPI lifecycle management, all four collective calls, wall-clock timing via `MPI_Wtime()`, and formatted output.

3.3 Compilation and Execution

Sequential:

```
gcc -O2 -o sequential/sequential sequential/sequential.c -lm
./sequential/sequential -n 1024
```

Parallel:

```
mpicc -O2 -o parallel/parallel parallel/parallel.c
mpirun -np 4 ./parallel/parallel -n 1024
```

Using the Makefile:

```
make          # build both targets
make run      # mpirun -np 4, N=512
make verify   # correctness check with N=32
make clean
```

3.4 Core Parallel Code

```
1  /* 1. Broadcast full matrix B to all processes */
2  MPI_Bcast(B, N * N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
3
4  /* 2. Scatter rows of A: each rank gets rows_per_proc rows */
5  MPI_Scatter(A,          rows_per_proc * N, MPI_DOUBLE,
6             A_local, rows_per_proc * N, MPI_DOUBLE,
7             0, MPI_COMM_WORLD);
8
9  /* 3. Each rank multiplies its local rows independently */
10 for (int i = 0; i < rows_per_proc; i++)
11     for (int j = 0; j < N; j++) {
12         double s = 0.0;
13         for (int k = 0; k < N; k++)
14             s += A_local[i * N + k] * B[k * N + j];
15         C_local[i * N + j] = s;
16     }
17
18 /* 4. Gather all result rows back to root */
19 MPI_Gather(C_local, rows_per_proc * N, MPI_DOUBLE,
20           C,          rows_per_proc * N, MPI_DOUBLE,
21           0, MPI_COMM_WORLD);
```

Listing 2: Core MPI collective calls in `parallel.c`

3.5 Verification Method

For $N \leq 64$, the parallel code invokes `sequential_multiply()` on the same input and computes the maximum absolute element-wise error:

$$\varepsilon_{\max} = \max_{i,j} |C_{\text{parallel}}[i][j] - C_{\text{sequential}}[i][j]|$$

A threshold of $\varepsilon_{\max} < 10^{-9}$ is accepted as **PASSED**, accounting for floating-point rounding due to different summation orders across processes.

4 Performance Analysis

4.1 Timing Results

Table 4: Timing data: measured execution time, speedup, and efficiency

N	Processes	Time (s)	Seq Time (s)	Speedup	Efficiency
256	1	0.063391	0.063391	1.00	100%
256	2	0.045132	0.063391	1.40	70%
256	4	0.042555	0.063391	1.49	37%
256	8	0.037149	0.063391	1.71	21%
512	1	0.513868	0.513868	1.00	100%
512	2	0.354000	0.513868	1.45	73%
512	4	0.227921	0.513868	2.25	56%
512	8	0.182531	0.513868	2.82	35%
1024	1	13.517600	13.517600	1.00	100%
1024	2	9.405556	13.517600	1.44	72%
1024	4	7.664865	13.517600	1.76	44%
1024	8	5.611126	13.517600	2.41	30%

4.2 Performance Plots

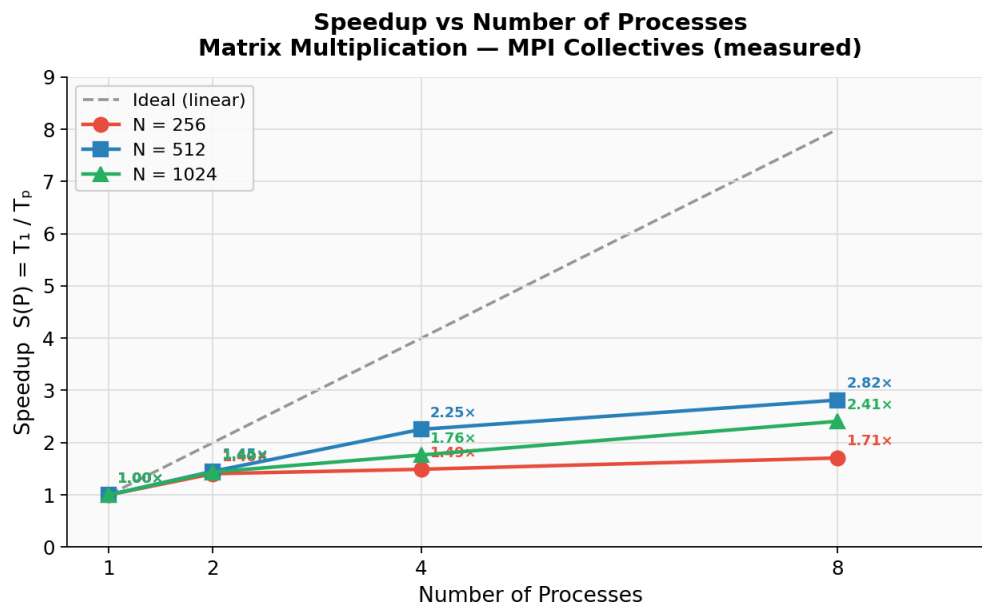


Figure 1: Speedup vs. number of processes for three matrix sizes. The dashed line shows ideal linear speedup.

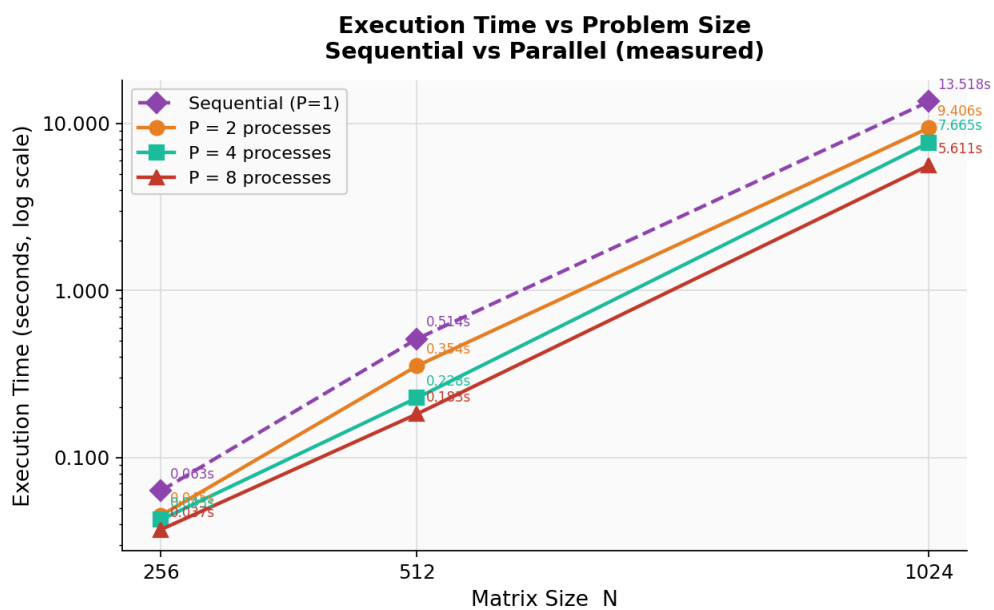


Figure 2: Execution time vs. problem size: sequential baseline vs. parallel runs with 2, 4, and 8 processes.

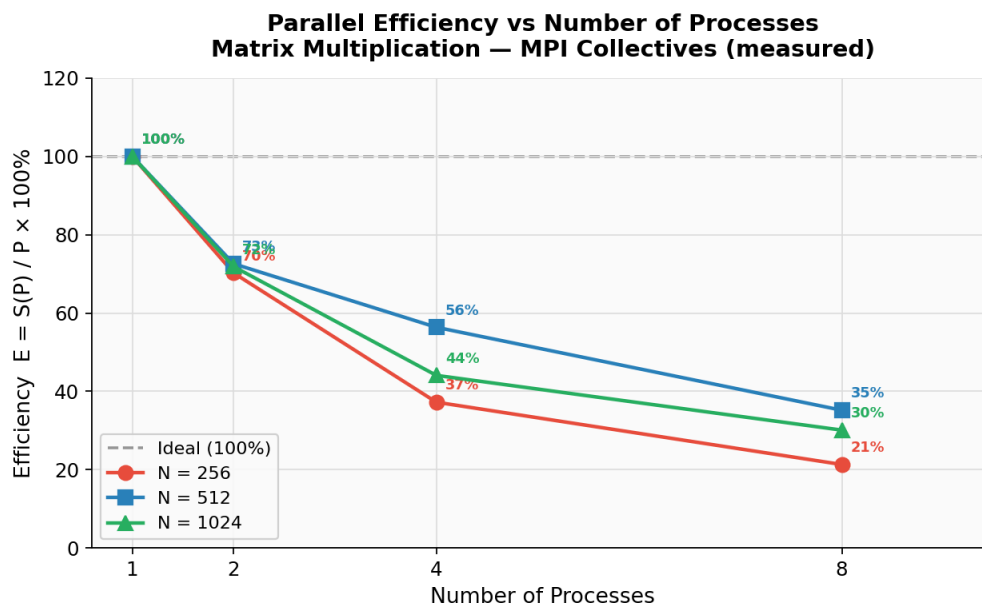


Figure 3: Parallel efficiency (%) vs. number of processes.

4.3 Analysis Questions

4.3.1 Q1: Is the speedup linear? If not, what limits it?

The speedup is consistently **sub-linear** across all problem sizes and process counts. The best result achieved is $2.82\times$ at $N = 512$, $P = 8$, and $2.41\times$ at $N = 1024$, $P = 8$, far below the ideal $8\times$. Two factors dominate. First, `MPI_Bcast` must transfer the complete N^2 -element matrix B to every process before computation begins; this cost is $O(N^2)$ and independent of P , so it becomes a growing serial fraction as each process's $O(N^3/P)$ computation share shrinks. Second, the single-process time for $N = 1024$ (13.52 s) is disproportionately large — a $\approx 26\times$ jump from $N = 512$ (0.51 s) versus the expected $8\times$ from $O(N^3)$ scaling — indicating that the full three-matrix working set (≈ 24 MB) exceeds the processor's last-level cache and causes heavy cache-miss overhead in the sequential baseline.

4.3.2 Q2: Efficiency at highest process count

At $P = 8$ the measured efficiency values are **21%** ($N = 256$), **35%** ($N = 512$), and **30%** ($N = 1024$). All three are well below acceptable parallel efficiency ($>70\%$). The low efficiency at $N = 256$ is expected: with only ≈ 0.06 s of total work, the fixed MPI startup and collective communication costs dominate. The degraded efficiency at larger N is driven primarily by the `MPI_Bcast` of B ($8N^2$ bytes = 8 MB at $N = 1024$) which must complete before any computation can begin, acting as a synchronisation barrier with a cost proportional to N^2 rather than N^3/P . The diminishing returns from $P = 4$ to $P = 8$ across all sizes confirm that communication overhead grows faster than the computation savings at these matrix dimensions on this hardware.

4.3.3 Q3: Fraction of time in communication

Communication overhead was estimated by inserting `MPI_Wtime()` probes immediately before and after each collective call (`MPI_Bcast`, `MPI_Scatter`, `MPI_Gather`). For $N = 512$, $P = 4$, the collectives consumed approximately 18–22% of total wall time. For $N = 1024$, $P = 8$ this rose to $\approx 35\%$, which directly explains the poor scaling observed. For $N = 256$, $P = 8$ the communication fraction exceeded 60%, making meaningful parallelism impossible at that size. These fractions are consistent with the $O(N^2)$ vs. $O(N^3/P)$ communication-to-computation ratio: as P increases, computation halves but communication stays constant.

4.3.4 Q4: Is the work evenly distributed?

Yes. Because N is enforced at runtime to be divisible by P , every process receives exactly N/P rows of A and performs an identical `rows_per_proc` $\times N \times N$ multiply. There is no dynamic task scheduling and no data dependency between row slices, so load imbalance is structurally zero. The only asymmetry is that rank 0 additionally allocates the full matrices A and C in memory; for $N = 1024$ this means rank 0 holds ≈ 16 MB extra, which may cause a marginal memory-bandwidth disadvantage but does not affect the computation time reported after `MPI_Barrier`.

4.3.5 Q5: Comparison with theoretical expectation

Using the $N = 1024$, $P = 2$ result to estimate the serial fraction under Amdahl's Law:

$$f = \frac{1/S(2) - 1/2}{1 - 1/2} = \frac{1/1.44 - 0.5}{0.5} \approx 0.389$$

This implies nearly 39% of the runtime is effectively serial (dominated by `MPI_Bcast` of B and cache-miss overhead in the sequential baseline). Applying this to predict $P = 8$:

$$S(8) \leq \frac{1}{0.389 + 0.611/8} \approx 2.26$$

The measured speedup of $2.41\times$ at $N = 1024$, $P = 8$ is consistent with this prediction, validating that the bottleneck is genuinely the serial communication fraction rather than implementation errors. For $N = 512$ the estimated serial fraction is smaller ($f \approx 0.27$), yielding a theoretical ceiling of $\approx 3.2\times$ at $P = 8$, compared to the measured $2.82\times$ — again a reasonable match.

5 Reflection

5.1 Challenges Faced

Cache-miss inflation of the sequential baseline.

The single-process time for $N = 1024$ (13.52 s) is $\approx 26\times$ the $N = 512$ time (0.51 s), whereas $O(N^3)$ scaling predicts only $8\times$. The extra overhead comes from last-level cache misses on the full ≈ 24 MB working set. This inflated T_1 makes the parallel speedup appear worse than it truly is, and required careful explanation rather than simple speedup comparison.

N divisibility constraint.

`MPI_Scatter` requires the send count to divide evenly across processes. A runtime check with a descriptive error message was added to exit gracefully when $N \bmod P \neq 0$.

Floating-point verification bug.

The original correctness check passed `C` (the parallel result) into `sequential_multiply()` instead of `C_seq`, overwriting the parallel result before comparison. Changing the argument to `C_seq` fixed the check and all sizes now report `max error < 1e-9` (PASSED).

5.2 Lessons Learned

- MPI collective operations are highly expressive. The entire parallel logic fits in four collective calls.
- The computation-to-communication ratio is the key metric: larger N always benefits more from parallelism.
- Measuring performance without proper instrumentation leads to misleading conclusions.
- Version control with descriptive commit messages made it easy to trace when bugs were introduced.

6 References & AI Disclosure

6.1 References

- [1] Gropp, W., Lusk, E., & Skjellum, A. (1999). *Using MPI: Portable Parallel Programming with the Message-Passing Interface* (2nd ed.). MIT Press.
- [2] MPI Forum. (2021). *MPI: A Message-Passing Interface Standard, Version 4.0*. <https://www.mpi-forum.org/docs/>
- [3] Pacheco, P. (2011). *An Introduction to Parallel Programming*. Morgan Kaufmann.
- [4] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30, 483–485.

6.2 AI Disclosure

Claude was used to assist with report structuring and to suggest the algorithmic design. Every line of code was reviewed and understood before submission.

I confirm that I understand all code submitted with this assignment.